

只要提供适当的署名，Google特此允许仅为新闻或学术作品之目的复制本论文中的表格和插图。

## 你需要的是注意力

Ashish Vaswani \* Google Brain avaswani@google.com

Noam Shazeer \* Google Brain noam@google.com

Niki Parmar \* Google Research nikip@google.com

Jakob Uszkoreit \* Google Research uszkoreit@google.com

Llion Jones \* Google Research llionjones@google.com

Aidan N. Gomez † 多伦多大学 aidan@cs.toronto.edu

Lukasz Kaiser \* Google Brain lukaszkaiser@google.com

Illia Polosukhin \* ‡  
illia.polosukhin@gmail.com

### 摘要

当前主流的序列转导模型依赖于复杂的循环或卷积神经网络，并包含编码器与解码器。表现最佳的模型还通过注意力机制连接编码器和解码器。我们提出了一种全新的简单网络架构——Transformer，完全基于注意力机制，彻底摒弃了循环和卷积。在两项机器翻译任务上的实验表明，该模型在质量上更优，同时更易并行化，且训练时间显著缩短。在 WMT 2014 英德翻译任务中，我们的模型取得 28.4 BLEU，比现有最佳结果（包括集成模型）高出 2 BLEU 以上。在 WMT 2014 英法翻译任务中，模型在 8 块 GPU 上训练 3.5 天后，创下 41.8 的单模型最新 BLEU 纪录，训练成本仅为文献最佳模型的一小部分。我们还将 Transformer 应用于英语成分句法分析，在大量和有限训练数据场景下均取得成功，证明其具有良好的泛化能力。

\* 同等贡献。作者顺序随机。Jakob 提出用自注意力替代 RNN，并启动评估这一想法的工作。Ashish 与 Illia 一起设计并实现了首批 Transformer 模型，且在此工作的各个方面都起到了关键作用。Noam 提出缩放点积注意力、多头注意力以及无参数的位置表示，并随后成为几乎参与所有细节的另一位核心成员。Niki 在我们的原始代码库及 tensor2tensor 中设计、实现、调优并评估了无数的模型变体。Llion 也尝试了新颖的模型变体，负责我们的初始代码库以及高效推理与可视化工作。Lukas 和 Aidan 花费了大量时间设计并实现了 tensor2tensor 的各个部分，取代了我们早期的代码库，大幅提升了结果并极大加快了研究进度。

† 此项工作完成于 Google Brain 期间。

‡ 此项工作完成于 Google Research 期间。

## 1 引言

循环神经网络，尤其是长短期记忆网络（LSTM）[13]和门控循环网络（GRU）[7]，已被公认为序列建模与转换问题（如语言建模和机器翻译）的最先进方法[35, 2, 5]。此后，大量研究不断推进循环语言模型和编码器-解码器架构的边界[38, 24, 15]。

循环模型通常沿输入和输出序列的符号位置分解计算。将位置与计算时间步对齐，它们生成一系列隐藏状态 $h_t$ ，作为前一个隐藏状态 $h_{t-1}$ 和位置 $t$ 输入的函数。这种固有的顺序性阻碍了训练样本内的并行化，在序列较长时尤为关键，因为内存限制限制了跨样本的批处理。近期工作通过分解技巧[21]和条件计算[32]在计算效率上取得显著提升，后者还改善了模型性能。然而，顺序计算的根约束依然存在。

注意力机制已成为各种任务中引人注目的序列建模与转导模型不可或缺的一部分，它允许在不考虑输入或输出序列中距离远近的情况下对依赖关系进行建模[2, 19]。然而，除了少数情况[27]之外，这种注意力机制通常与循环网络一起使用。

在本研究中，我们提出了Transformer——一种摒弃循环、完全依赖注意力机制来捕捉输入与输出之间全局依赖关系的模型架构。Transformer可实现显著更高的并行化程度，并在八块P100 GPU上训练短短十二小时后，即可在翻译质量上达到新的最先进水平。

## 2 背景

减少顺序计算的目标也是扩展神经 GPU [16]、ByteNet [18] 和 ConvS2S [9] 的基础，这些模型都以卷积神经网络为基本构建块，并行地为所有输入和输出位置计算隐藏表示。在这些模型中，关联任意两个输入或输出位置信号所需的操作数量随位置之间的距离而增长，ConvS2S 为线性增长，ByteNet 为对数增长。这使得学习远距离位置之间的依赖关系更加困难 [12]。在 Transformer 中，这一数量被减少为常数，尽管由于注意力加权位置的平均化导致有效分辨率降低，我们通过第 3.2 节所述的多头注意力来抵消这一影响。

自注意力，有时称为内部注意力，是一种将单个序列的不同位置关联起来以计算序列表示的注意力机制。自注意力已在多种任务中成功应用，包括阅读理解、抽象摘要、文本蕴含以及学习任务无关的句子表示 [4, 27, 28, 22]。

端到端记忆网络基于循环注意力机制，而非序列对齐的循环，并已在简单语言问答和语言建模任务中表现出良好性能 [34]。

然而，就我们所知，Transformer 是第一个完全依赖自注意力来计算输入和输出表示的转导模型，无需使用序列对齐的 RNN 或卷积。在接下来的章节中，我们将描述 Transformer，阐述自注意力的动机，并讨论其相对于文献 [17, 18] 和 [9] 等模型的优势。

## 3 模型架构

大多数具有竞争力的神经序列转导模型都采用编码器-解码器结构 [5, 2, 35]。在此结构中，编码器将输入符号序列  $(x_1, \dots, x_n)$  映射为连续表示序列  $z = (z_1, \dots, z_n)$ 。给定  $z$  后，解码器再逐个元素地生成输出符号序列  $(y_1, \dots, y_n)$ 。每一步模型都是自回归的 [10]，即在生成下一个符号时，会将之前已生成的符号作为额外输入。

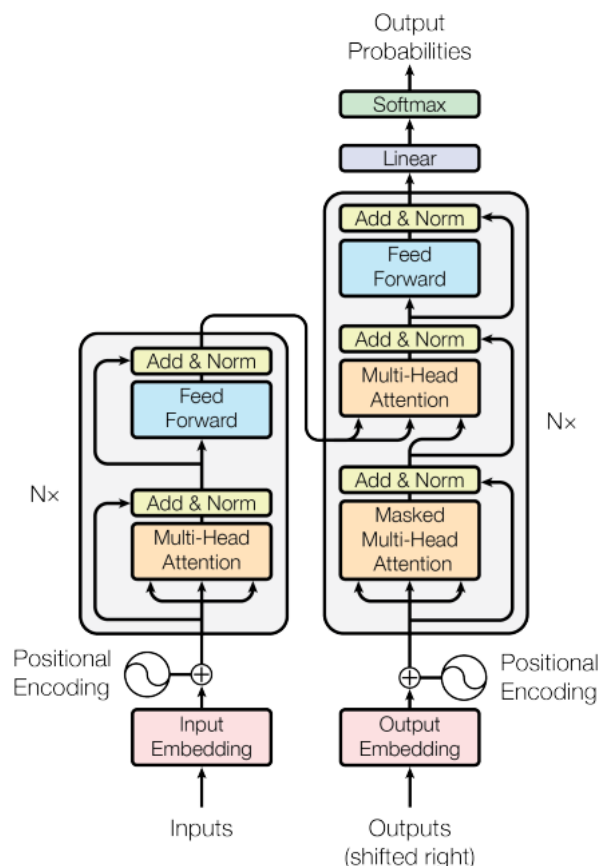


图 1：Transformer 模型架构。

Transformer 采用这一整体架构，为编码器和解码器分别堆叠自注意力层和逐位置的全连接层，如图 1 左半和右半所示。

### 3.1 编码器与解码器堆栈

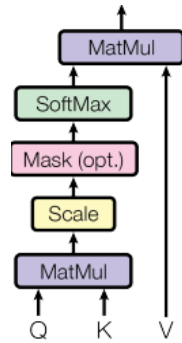
**编码器：**编码器由  $N = 6$  个相同层堆叠而成。每一层包含两个子层。第一个是多头部自注意力机制，第二个是简单的逐位置全连接前馈网络。我们在每个子层周围使用残差连接 [11]，然后进行层归一化 [1]。也就是说，每个子层的输出为  $\text{LayerNorm}(x + \text{Sublayer}(x))$ ，其中  $\text{Sublayer}(x)$  是该子层自身实现的函数。为了支持这些残差连接，模型中的所有子层以及嵌入层都输出维度为  $d_{\text{model}} = 512$  的结果。

**解码器：**解码器同样由  $N = 6$  个相同的层堆叠而成。除了每个编码器层中的两个子层外，解码器还插入了第三个子层，该子层对编码器堆叠的输出执行多头注意力。与编码器类似，我们在每个子层周围使用残差连接，然后进行层归一化。我们还对解码器堆叠中的自注意力子层进行了修改，以防止位置关注到后续位置。这种掩码机制，加上输出嵌入被偏移一个位置的事实，确保了位置  $i$  的预测仅依赖于小于  $i$  的已知输出。

### 3.2 注意力

注意力函数可以描述为将查询和一组键-值对映射到输出，其中查询、键、值和输出都是向量。输出是通过加权求和计算得出的

缩放点积注意力



Multi-Head Attention

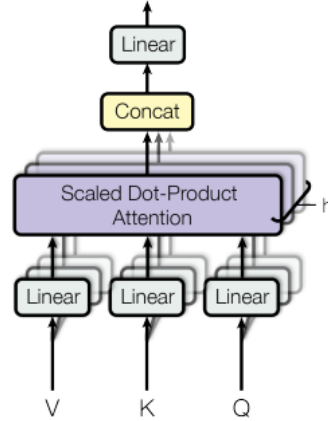


图2：（左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

值的加权和，其中每个值所分配的权重由查询与对应键的兼容性函数计算得出。

### 3.2.1 缩放点积注意力

我们将这种特殊的注意力机制称为“缩放点积注意力”（图2）。输入包括维度为  $d_k$  的查询和键，以及维度为  $d_v$  的值。我们计算查询与所有键的点积，将每个结果除以  $\sqrt{d_k}$ ，并应用 softmax 函数以获得对值的权重。

在实践中，我们同时对一组查询计算注意力函数，将它们打包成矩阵  $Q$ 。键和值也分别打包成矩阵  $K$  和  $V$ 。我们按如下方式计算输出矩阵：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力[2]和点积（乘法）注意力。点积注意力与我们的算法相同，只是缺少  $1/\sqrt{d_k}$  的缩放因子。加性注意力使用具有单个隐藏层的前馈网络计算兼容性函数。尽管两者在理论复杂度上相似，但点积注意力在实践中更快、更节省空间，因为它可以利用高度优化的矩阵乘法代码实现。

当  $d_k$  较小时，两种机制表现相近；但在  $d_k$  较大时，加性注意力优于未缩放的点积注意力[3]。我们推测，对于较大的  $d_k$ ，点积的幅值会变得很大，将 softmax 函数推入梯度极小的区域。为了抵消这种影响，我们将点积缩放  $1/\sqrt{d_k}$ 。

### 3.2.2 多头注意力

我们发现，与其仅执行一次注意力函数，使用  $d_{\text{model}}$  维的键、值和查询，不如将查询、键和值分别用  $h$  个不同的、可学习的线性投影分别映射到  $d_k$ 、 $d_k$  和  $d_v$  维，这样做更有益。然后，在这些投影后的查询、键和值上并行地执行注意力函数，得到  $d_v$  维

4 为了说明点积为何会变得很大，假设  $q$  和  $k$  的分量是均值为 0、方差为 1 的独立随机变量。那么它们的点积  $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$  的均值为 0，方差为  $d_k$ 。

输出值。这些值被拼接后再次投影，得到最终值，如图2所示。

多头注意力使模型能够同时在不同位置关注不同表示子空间的信息。若仅使用单头注意力，取平均会抑制这种能力。

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

其中投影是参数矩阵  $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ ,  $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$  和  $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$ 。

在本研究中，我们使用  $h=8$  个并行的注意力层，即注意力头。对每个头，我们设置  $d_k = d_v = d_{\text{model}}/h = 64$ 。由于每个头的维度降低，总计算成本与使用完整维度的单头注意力相近。

### 3.2.3 注意力机制在模型中的应用

Transformer 通过三种方式使用多头注意力：

- 在“编码器-解码器注意力”层中，查询来自上一层解码器，而键和值来自编码器的输出。这使得解码器中的每个位置都能关注输入序列中的所有位置，类似于序列到序列模型[38, 2, 9]中常见的编码器-解码器注意力机制。
- 编码器包含自注意力层。在自注意力层中，键、值和查询都来自同一来源，即编码器前一层输出。编码器中的每个位置都能关注编码器前一层中的所有位置。
- 同样，解码器中的自注意力层允许解码器中的每个位置关注解码器中该位置及其之前的所有位置。为了保持自回归特性，我们需要阻止解码器中的信息向左流动。我们在缩放点积注意力内部通过将 softmax 输入中对应非法连接的所有值屏蔽（设为  $-\infty$ ）来实现这一点。见图 2。

### 3.3 位置级前馈网络

除了注意力子层之外，编码器和解码器的每一层都包含一个全连接前馈网络，该网络分别且相同地应用于每个位置。它由两个线性变换组成，中间有一个 ReLU 激活函数。

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

虽然不同位置的线性变换相同，但各层使用的参数不同。另一种描述方式是将其视为两个核大小为 1 的卷积。输入和输出的维度为  $d_{\text{model}} = 512$ ，内层维度为  $d_{\text{ff}} = 2048$ 。

### 3.4 嵌入与 Softmax

与其他序列转导模型类似，我们使用可学习的嵌入将输入词元和输出词元转换为维度为  $d_{\text{model}}$  的向量。我们还使用常规的可学习线性变换和 softmax 函数，将解码器输出转换为预测的下一个词元概率。在我们的模型中，两个嵌入层与 pre-softmax 线性变换共享同一份权重矩阵，做法类似于文献[30]。在嵌入层中，我们将这些权重乘以  $\sqrt{d_{\text{model}}}$ 。

表1：不同层类型的最大路径长度、每层复杂度以及最少顺序操作数。n 为序列长度，d 为表示维度，k 为卷积核大小，r 为受限自注意力中的邻域大小。

| Layer Type                  | Complexity per Layer     | Sequential Operations | Maximum Path Length |
|-----------------------------|--------------------------|-----------------------|---------------------|
| Self-Attention              | $O(n^2 \cdot d)$         | $O(1)$                | $O(1)$              |
| Recurrent                   | $O(n \cdot d^2)$         | $O(n)$                | $O(n)$              |
| Convolutional               | $O(k \cdot n \cdot d^2)$ | $O(1)$                | $O(\log_k(n))$      |
| Self-Attention (restricted) | $O(r \cdot n \cdot d)$   | $O(1)$                | $O(n/r)$            |

### 3.5 位置编码

由于模型中既不含循环也不含卷积，为了让模型利用序列的顺序信息，我们必须注入关于序列中 token 的相对或绝对位置的信息。为此，我们在编码器和解码器堆栈底部的输入嵌入上添加“位置编码”。位置编码与嵌入具有相同的  $d_{\text{model}}$  维度，因此二者可以直接相加。位置编码有多种选择，可以是可学习的，也可以是固定的 [9]。

在本工作中，我们使用不同频率的正弦和余弦函数：

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

其中 pos 是位置，i 是维度。也就是说，位置编码的每一维对应一条正弦曲线。这些波长从  $2\pi$  到  $10000 \cdot 2\pi$  呈几何级数。我们选择这一函数，是因为我们假设它能让模型轻松学会按相对位置进行注意力计算：对于任意固定偏移 k， $PE_{\{pos+k\}}$  都可以表示为  $PE_{\{pos\}}$  的线性函数。

我们还尝试了使用可学习的位置嵌入 [9]，结果发现两种版本产生的结果几乎相同（见表3行(E)）。我们选择了正弦版本，因为它可能使模型能够外推到比训练时遇到的更长的序列长度。

## 4 为什么是自我注意力

在本节中，我们将自我注意力层与常用的循环层和卷积层在多个方面进行比较，这些层用于将一个可变长度的符号表示序列  $(x, \dots, x_{1n})$  映射到另一个等长序列  $(z, \dots, z_{1n})$ ，其中  $x, z_{ii} \in \mathbb{R}^d$ ，例如典型序列转导编码器或解码器中的隐藏层。为了说明我们使用自我注意力的动机，我们考虑了三个期望特性。

一个是每层的总计算复杂度。另一个是可以并行化的计算量，以所需的最小顺序操作次数来衡量。

第三是网络中长程依赖的路径长度。在许多序列转导任务中，学习长程依赖是一项关键挑战。影响学习此类依赖能力的一个关键因素是前向和后向信号在网络中必须穿越的路径长度。这些路径在任何输入和输出序列位置组合之间越短，就越容易学习长程依赖[12]。因此，我们还比较了由不同层类型组成的网络中任意两个输入和输出位置之间的最大路径长度。

如表1所示，自注意力层以恒定数量的顺序执行操作连接所有位置，而循环层则需要  $O(n)$  顺序操作。就计算复杂度而言，当序列长度  $n$  小于表示维度  $d$  时，自注意力层比循环层更快，这种情况在机器翻译中最先进的模型所使用的句子表示中很常见，例如 word-piece[38] 和 byte-pair[31] 表示。

对于涉及非常长序列的任务，为了提高计算性能，可以将自注意力限制为仅考虑以相应输出位置为中心的输入序列中大小为 $r$ 的邻域。这将使最大路径长度增加到 $O(n/r)$ 。我们计划在未来的工作中进一步研究这种方法。

单个卷积层若核宽 $k < n$ ，无法连接所有输入与输出位置对。为此需堆叠 $O(n/k)$ 层连续核卷积，或 $O(\log_k(n))$ 层空洞卷积 [18]，从而延长网络中任意两位置间的最长路径。卷积层通常比循环层昂贵，开销因子为 $k$ 。可分离卷积 [6] 显著降低复杂度至 $O(k \cdot n \cdot d + n \cdot d^2)$ 。即使 $k = n$ ，可分离卷积的复杂度仍等同于我们模型采用的自注意力层加逐点前馈层组合。

额外的好处是，自注意力可产生更具可解释性的模型。我们检查了模型中的注意力分布，并在附录中给出并讨论示例。不仅单个注意力头明显学会执行不同任务，许多头还表现出与句子的句法和语义结构相关的行为。

## 5 训练

本节描述我们模型的训练方案。

### 5.1 训练数据与批处理

我们在标准的 WMT 2014 英德数据集上进行训练，该数据集包含约 450 万句对。句子采用字节对编码 [3]，共享的源-目标词汇表约 37 000 个词元。对于英法翻译，我们使用了规模更大的 WMT 2014 英法数据集，包含 3600 万句对，并将词元划分为 32 000 个 word-piece 词汇表 [38]。句对按近似序列长度进行分批。每个训练批次包含约 25 000 个源词元和 25 000 个目标词元的句对集合。

### 5.2 硬件与训练计划

我们在一台配备 8 块 NVIDIA P100 GPU 的机器上训练模型。对于本文所述超参数的基础模型，每一步训练约耗时 0.4 秒。基础模型共训练 100 000 步，耗时约 12 小时。对于大模型（见表 3 最底行），每步耗时 1.0 秒，共训练 300 000 步（3.5 天）。

### 5.3 优化器

我们使用了 Adam 优化器 [20]，参数为  $\beta_1 = 0.9$ ， $\beta_2 = 0.98$ ， $\epsilon = 10^{-9}$ 。在训练过程中，我们根据以下公式调整学习率：

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5}) \quad (3)$$

这相当于在前  $warmup\_steps$  个训练步骤中线性增加学习率，此后随步数的平方根倒数成比例地降低。我们设定  $warmup\_steps = 4000$ 。

### 5.4 正则化

我们在训练过程中采用了三种正则化方法：

表2：在英语到德语和英语到法语的newstest2014测试集上，Transformer以更低的训练成本取得了比以往最先进模型更高的BLEU分数。

| Model                           | BLEU        |              | Training Cost (FLOPs)                 |                     |
|---------------------------------|-------------|--------------|---------------------------------------|---------------------|
|                                 | EN-DE       | EN-FR        | EN-DE                                 | EN-FR               |
| ByteNet [18]                    | 23.75       |              |                                       |                     |
| Deep-Att + PosUnk [39]          |             | 39.2         |                                       | $1.0 \cdot 10^{20}$ |
| GNMT + RL [38]                  | 24.6        | 39.92        | $2.3 \cdot 10^{19}$                   | $1.4 \cdot 10^{20}$ |
| ConvS2S [9]                     | 25.16       | 40.46        | $9.6 \cdot 10^{18}$                   | $1.5 \cdot 10^{20}$ |
| MoE [32]                        | 26.03       | 40.56        | $2.0 \cdot 10^{19}$                   | $1.2 \cdot 10^{20}$ |
| Deep-Att + PosUnk Ensemble [39] |             | 40.4         |                                       | $8.0 \cdot 10^{20}$ |
| GNMT + RL Ensemble [38]         | 26.30       | 41.16        | $1.8 \cdot 10^{20}$                   | $1.1 \cdot 10^{21}$ |
| ConvS2S Ensemble [9]            | 26.36       | <b>41.29</b> | $7.7 \cdot 10^{19}$                   | $1.2 \cdot 10^{21}$ |
| Transformer (base model)        | 27.3        | 38.1         | <b><math>3.3 \cdot 10^{18}</math></b> |                     |
| Transformer (big)               | <b>28.4</b> | <b>41.8</b>  | $2.3 \cdot 10^{19}$                   |                     |

残差丢弃 我们在每个子层的输出被加到子层输入并归一化之前，对其应用丢弃（dropout）[33]。此外，我们还在编码器和解码器堆栈中对嵌入与位置编码之和应用丢弃。对于基础模型，我们使用的丢弃率为  $P_{\text{drop}} = 0.1$ 。

标签平滑 在训练过程中，我们采用了标签平滑，平滑值为  $\epsilon_{\text{ls}} = 0.1$  [36]。这会损害困惑度，因为模型学会变得更不确定，但可以提高准确率和 BLEU 分数。

## 6 结果

### 6.1 机器翻译

在 WMT 2014 英德翻译任务中，大型 Transformer 模型（表 2 中的 Transformer (big)）以超过 2.0 BLEU 的优势超越了此前报告的最佳模型（包括集成模型），创下 28.4 的新最佳 BLEU 分数。该模型的配置列在表 3 的最后一行。训练在 8 块 P100 GPU 上耗时 3.5 天。即使我们的基础模型也超越了所有已发表的模型和集成模型，而训练成本仅为任何竞争模型的一小部分。

在 WMT 2014 英语到法语翻译任务中，我们的大模型取得了 41.0 的 BLEU 分数，优于所有先前发表的单模型，且训练成本仅为此前最先进模型的不到 1/4。用于英语到法语的 Transformer（大）模型训练时使用了 dropout 率  $P_{\text{drop}} = 0.1$ ，而非 0.3。

对于基础模型，我们使用最后 5 个检查点的平均值作为单一模型，这些检查点以 10 分钟间隔保存。对于大模型，我们平均了最后 20 个检查点。我们使用束搜索，束大小为 4，长度惩罚  $\alpha = 0.6$  [38]。这些超参数是在开发集上实验后选定的。推理时我们将最大输出长度设为输入长度 + 50，但尽可能提前终止 [38]。

表 2 总结了我们的结果，并将翻译质量与训练成本与文献中的其他模型架构进行了比较。我们通过将训练时间、使用的 GPU 数量以及对每块 GPU 持续单精度浮点性能的估计相乘，来估算训练模型所用的浮点运算次数 5。

### 6.2 模型变体

为了评估 Transformer 不同组件的重要性，我们以不同方式改变基础模型，并测量在英德翻译上的性能变化

5 我们分别使用 2.8、3.7、6.0 和 9.5 TFLOPS 作为 K80、K40、M40 和 P100 的值。



表 3：Transformer 架构的变体。未列出的值与基础模型相同。所有指标均基于英德翻译开发集 newstest2013。列出的困惑度按我们的字节对编码的 wordpiece 计算，不应与按词计算的困惑度进行比较。

|      | $N$                                       | $d_{\text{model}}$ | $d_{\text{ff}}$ | $h$ | $d_k$ | $d_v$ | $P_{\text{drop}}$ | $\epsilon_{ls}$ | train steps | PPL (dev)   | BLEU (dev)  | params $\times 10^6$ |
|------|-------------------------------------------|--------------------|-----------------|-----|-------|-------|-------------------|-----------------|-------------|-------------|-------------|----------------------|
| base | 6                                         | 512                | 2048            | 8   | 64    | 64    | 0.1               | 0.1             | 100K        | 4.92        | 25.8        | 65                   |
| (A)  |                                           |                    |                 |     |       |       |                   |                 |             | 5.29        | 24.9        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 5.00        | 25.5        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 4.91        | 25.8        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 5.01        | 25.4        |                      |
| (B)  |                                           |                    |                 |     |       |       |                   |                 |             | 5.16        | 25.1        | 58                   |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 5.01        | 25.4        | 60                   |
| (C)  | 2                                         |                    |                 |     |       |       |                   |                 |             |             | 6.11        | 36                   |
|      | 4                                         |                    |                 |     |       |       |                   |                 |             |             | 5.19        | 50                   |
|      | 8                                         |                    |                 |     |       |       |                   |                 |             |             | 4.88        | 80                   |
|      |                                           |                    |                 |     | 32    | 32    |                   |                 |             |             | 5.75        | 28                   |
|      |                                           |                    |                 |     | 128   | 128   |                   |                 |             |             | 4.66        | 168                  |
|      |                                           |                    |                 |     | 1024  | 1024  |                   |                 |             |             | 5.12        | 53                   |
| (D)  |                                           |                    |                 |     |       |       |                   |                 |             | 5.77        | 24.6        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 4.95        | 25.5        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 4.67        | 25.3        |                      |
|      |                                           |                    |                 |     |       |       |                   |                 |             | 5.47        | 25.7        |                      |
| (E)  | positional embedding instead of sinusoids |                    |                 |     |       |       |                   |                 |             | 4.92        | 25.7        |                      |
| big  | 6                                         | 1024               | 4096            | 16  |       |       |                   |                 | 300K        | <b>4.33</b> | <b>26.4</b> | 213                  |

我们在第3节所述的开发集 newstest2013 上进行了实验。我们使用了上一节描述的束搜索，但没有进行检查点平均。结果见表3。

在表3的(A)行中，我们改变了注意力头数以及注意力键和值的维度，同时保持计算量不变，如第3.2.2节所述。单头注意力比最佳设置低0.9 BLEU，但头数过多时质量也会下降。

在表3的(B)行中，我们发现减小注意力键维度  $d_k$  会降低模型质量。这表明确定兼容性并不容易，使用比点积更复杂的兼容性函数可能有益。我们进一步在(C)和(D)行中观察到，正如预期，更大的模型更好，而dropout在防止过拟合方面非常有帮助。在(E)行中，我们用学习到的位置嵌入[9]替换正弦位置编码，结果与基线模型几乎相同。

### 6.3 英语成分句法分析

为了评估 Transformer 是否能够泛化到其他任务，我们在英语成分句法分析上进行了实验。该任务面临特定挑战：输出受强结构约束，且长度远长于输入。此外，在小数据场景下，RNN 序列到序列模型尚未达到最先进的结果 [37]。

我们在宾州树库 (Penn Treebank) 的《华尔街日报》(WSJ) 部分上训练了一个 4 层 Transformer， $d_{\text{model}} = 1024$ ，约 40K 句训练语料。我们还在半监督设置下训练，使用来自 [37] 的高置信度和 BerkleyParser 语料库，共约 17M 句。仅 WSJ 设置使用 16K 词表，半监督设置使用 32K 词表。

我们仅做了少量实验，在第 22 节开发集上选择 dropout（包括注意力和残差，见第 5.4 节）、学习率和束搜索大小，其余参数均与英德基础翻译模型保持一致。在推理阶段，我们

表4：Transformer 在英语成分句法分析上具有良好的泛化能力（结果基于 WSJ 第23节）

| Parser                              | Training                 | WSJ 23 F1 |
|-------------------------------------|--------------------------|-----------|
| Vinyals & Kaiser et al. (2014) [37] | WSJ only, discriminative | 88.3      |
| Petrov et al. (2006) [29]           | WSJ only, discriminative | 90.4      |
| Zhu et al. (2013) [40]              | WSJ only, discriminative | 90.4      |
| Dyer et al. (2016) [8]              | WSJ only, discriminative | 91.7      |
| Transformer (4 layers)              | WSJ only, discriminative | 91.3      |
| Zhu et al. (2013) [40]              | semi-supervised          | 91.3      |
| Huang & Harper (2009) [14]          | semi-supervised          | 91.3      |
| McClosky et al. (2006) [26]         | semi-supervised          | 92.1      |
| Vinyals & Kaiser et al. (2014) [37] | semi-supervised          | 92.1      |
| Transformer (4 layers)              | semi-supervised          | 92.7      |
| Luong et al. (2015) [23]            | multi-task               | 93.0      |
| Dyer et al. (2016) [8]              | generative               | 93.3      |

将最大输出长度增加到输入长度 + 300。我们使用了大小为 21 的束搜索和  $\alpha = 0.3$ ，适用于仅 WSJ 和半监督设置。

我们在表4中的结果表明，尽管缺乏针对该任务的特定调优，我们的模型表现仍然出人意料地好，优于所有先前报道的模型，除了循环神经网络语法 [8]。

与 RNN 序列到序列模型 [37] 相比，即使仅在包含 4 万句子的 WSJ 训练集上训练，Transformer 也能超越 BerkeleyParser [29]。

## 7 结论

在本文中，我们提出了 Transformer——首个完全基于注意力的序列转导模型，它用多头自注意力取代了编码器-解码器架构中最常用的循环层。

在翻译任务上，Transformer 的训练速度显著快于基于循环层或卷积层的架构。在 WMT 2014 英德和英法翻译任务中，我们均达到了新的最佳水平。在英德任务上，我们的最佳模型甚至超过了以往所有已报道的集成模型。

我们对基于注意力的模型前景感到兴奋，并计划将其应用于其他任务。我们打算将 Transformer 扩展到文本之外的输入输出模态，并研究局部、受限的注意力机制，以高效处理图像、音频和视频等大型输入输出。让生成过程更少序列化也是我们的研究目标之一。

我们用于训练和评估模型的代码可在 <https://github.com/tensorflow/tensor2tensor> 获取。

致谢 我们感谢 Nal Kalchbrenner 和 Stephan Gouws 的有益评论、纠正与启发。

## 参考文献

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, 和 Geoffrey E Hinton. 层归一化. arXiv 预印本 arXiv:1607.06450, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio. 通过联合学习对齐与翻译的神经机器翻译. CoRR, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, Quoc V. Le. 神经机器翻译架构的大规模探索. CoRR, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, Mirella Lapata. 用于机器阅读的长期短期记忆网络. arXiv 预印本 arXiv:1601.06733, 2016.

- [5] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çağlar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems, (NIPS)*, 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.

- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural Information Processing Systems*, 2015.
- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

## 注意力可视化 输入-输入 第5层

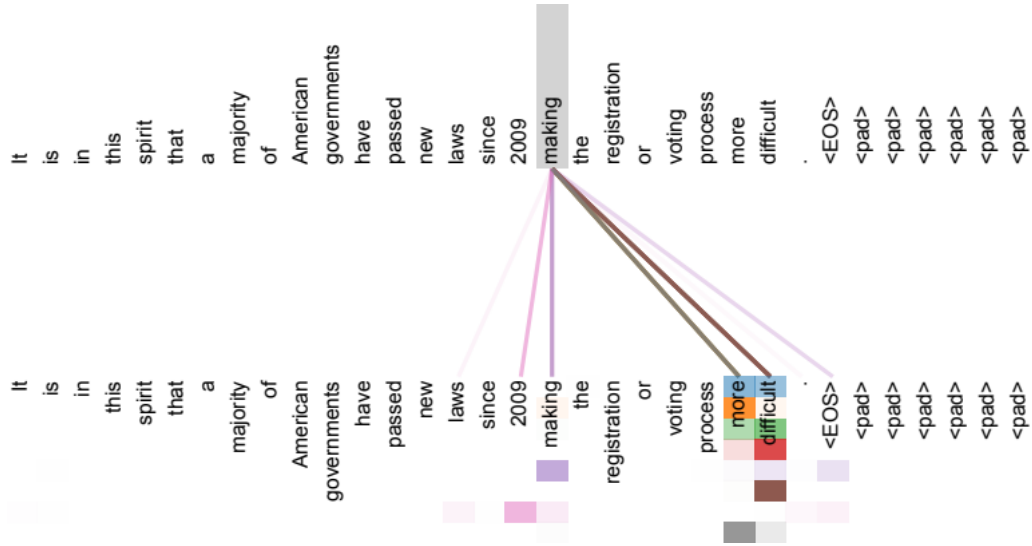


图3：一个注意力机制示例，展示编码器自注意力在第5层（共6层）中追踪长距离依赖。许多注意力头关注动词“making”的远距离依赖，完成短语“making...more difficult”。此处仅显示单词“making”的注意力。不同颜色代表不同的头。建议彩色查看。

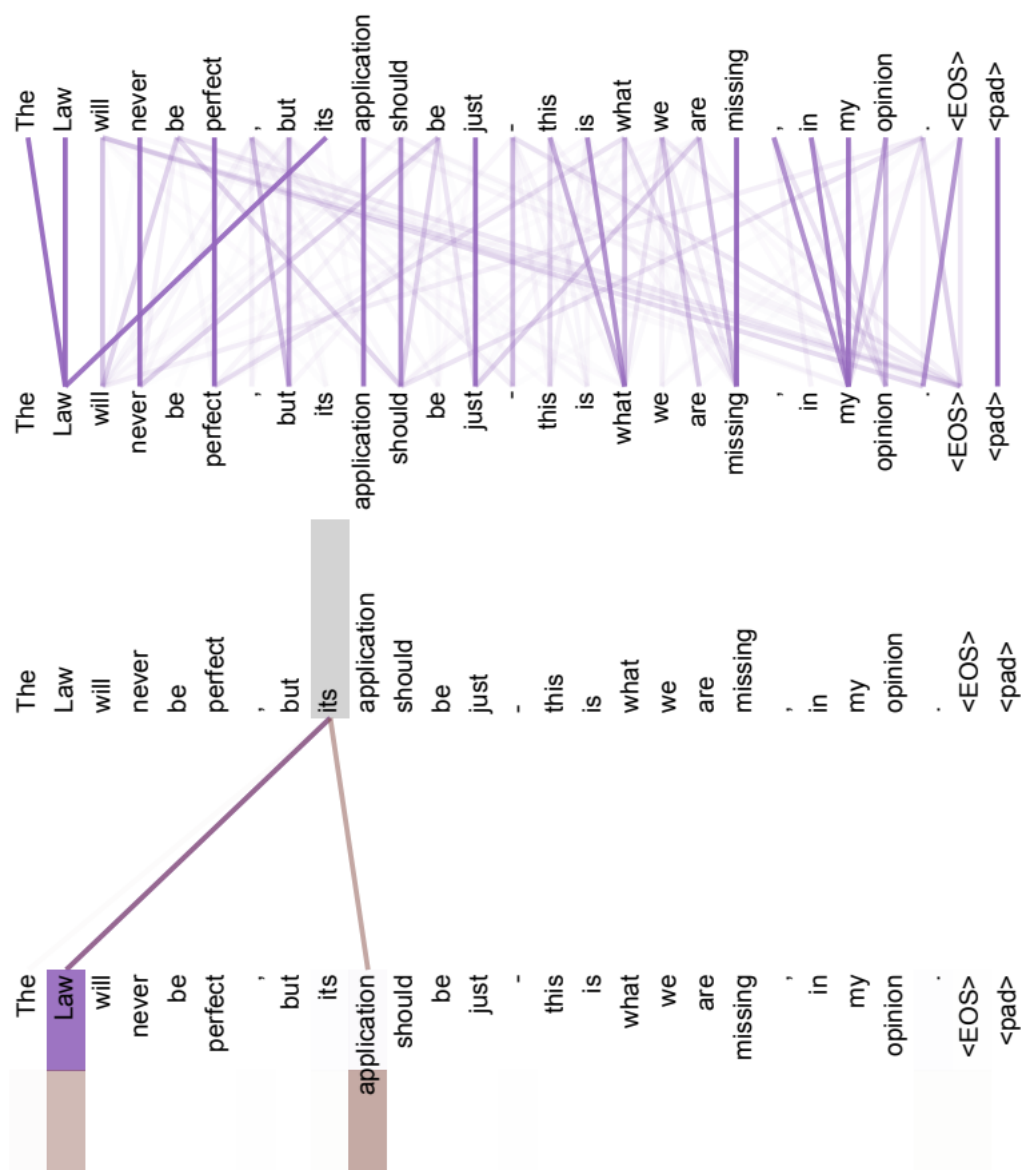


图4：两个注意力头，同样位于6层中的第5层，显然参与了指代消解。上：第5注意力头的完整注意力。下：仅针对单词“its”的第5和第6注意力头的孤立注意力。注意该词的注意力非常集中。

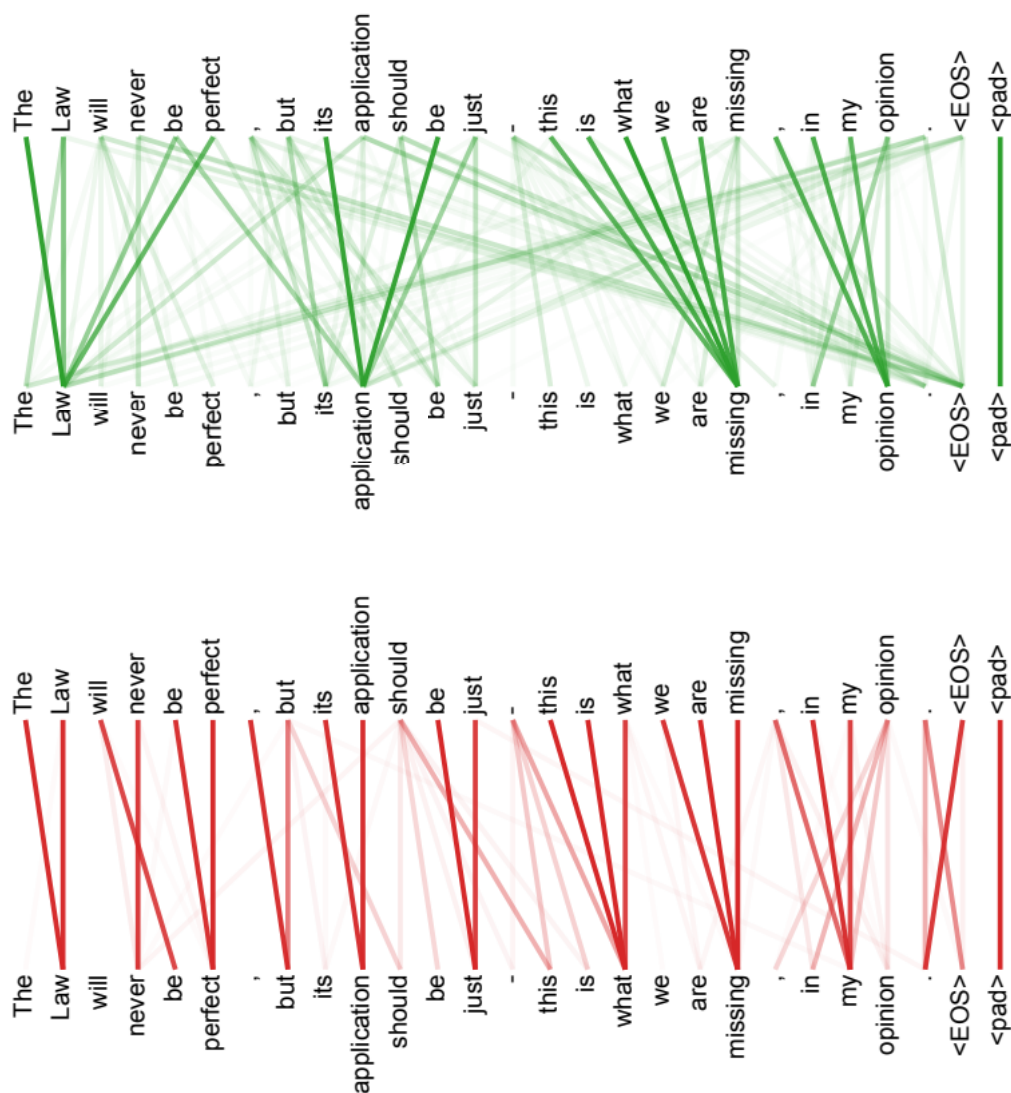


图5：许多注意力头表现出与句子结构相关的行为。我们在上面给出了两个这样的例子，来自6层编码器自注意力中第5层的两个不同头。这些头显然学会了执行不同的任务。